

6CS005
High Performance Computing
Week 6 Workshop
OpenMP Multithreading

Student Number: 2408869

Name: Narayan Lohani

Group: L6CG15

Table of Contents

Question 1	3
Question 2	4
Question 3	5
Question 4	6
Question 5	7
Task 1	11
Task 2	13

Question 1

1. Write a OpenMP program to print out “Hello World!” with the default number of threads which is usually the number of CPU processor cores that are on your computer system:
 - a. Enter and run the program to see how many processor cores are on the system you are using.

In the following execution, 10 processor cores are used to parallelism.

```
narayan@narayan workshop % gcc-15 -fopenmp 1.c -o o/1
narayan@narayan workshop % o/1
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
narayan@narayan workshop %
```

- b. Modify the program so that it runs with exactly 10 threads.

To make the program run with exactly 10 threads “num_threads()” clause is used with parallel construct.

```
hpc_herald - 1.c
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      #pragma omp parallel num_threads(10)
6          printf("Hello world from OpenMP!\n");
7      return 0;
8  }
9
```

```
narayan@narayan workshop % gcc-15 -fopenmp 1.c -o o/1
narayan@narayan workshop % o/1
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
Hello world from OpenMP!
narayan@narayan workshop %
```

Question 2

2. Write a multithreaded OpenMP program to print out all the prime numbers from 1 to 1000 using the default number of threads.

To solve this program, for clause is used with parallel construct. As thread number are not defined explicitly, the number of threads is assigned by the availability of logical processors. The parallel construct divides threads are prints the prime numbers.

```
hpc_herald - 2.c

1  #include <stdio.h>
2  int main() {
3      int i, c;
4      printf("Prime numbers between 1 and 1000 are :\n");
5      #pragma omp parallel for
6      for (i = 1; i <= 1000; i++) {
7          for (c = 2; c <= i - 1; c++) {
8              if (i % c == 0)
9                  break;
10         }
11         if (c == i)
12             printf("%d\t", i);
13     }
14     return 0;
15 }
16
```

```
• narayan@narayan workshop % gcc-15 -fopenmp 2.c -o o/2
• narayan@narayan workshop % o/2
Prime numbers between 1 and 1000 are :
101 103 107 109 113 127 131 137 139 149 307 401 311 503 313 409 601 509 701 521 419 607 4
21 709 523 431 613 809 433 541 617 811 547 619 439 557 2 3 5 821 631 449 7 11 1
3 457 17 641 461 19 823 265 643 463 827 151 467 647 829 479 317 331 487 653 719 337 8
39 727 349 353 359 563 367 733 569 373 987 739 571 23 743 577 911 31 587 751 919 157 4
91 163 929 167 757 173 599 499 179 181 937 761 191 659 193 853 197 199 769 941 661 857 3
79 773 673 947 859 677 787 37 953 41 43 47 53 863 683 967 691 383 211 797 389 971 2
23 227 877 397 59 229 61 67 233 71 977 73 239 79 881 83 241 89 97 251 983 257 8
83 269 271 887 991 997 277 281 283 293
```

Question 3

3. Convert the above program so that it uses exactly 5 threads and each thread prints out their thread ID when they print out their prime number.

To use exactly 5 threads, `num_threads(5)` clause is used. And to print the threadID, routine of omp namely, `omp_get_thread_num()` is used.

```
hpc_herald - 3.c
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main() {
5      int i, c;
6      int tid;
7      printf("Prime numbers between 1 and 1000 are :\n");
8      #pragma omp parallel for num_threads(5)
9      for (i = 1; i <= 1000; i++) {
10         for (c = 2; c <= i - 1; c++) {
11             if (i % c == 0)
12                 break;
13         }
14         if (c == i) {
15             tid = omp_get_thread_num();
16             printf("%d thread found %d\n", tid, i);
17         }
18     }
19     return 0;
20 }
21
```

```
narayan@narayan workshop % gcc-15 -fopenmp 3.c -o o/3
narayan@narayan workshop % o/3
Prime numbers between 1 and 1000 are :
1 thread found 211
1 thread found 223
1 thread found 227
0 thread found 2
0 thread found 3
0 thread found 5
0 thread found 7
0 thread found 11
0 thread found 13
0 thread found 17
0 thread found 19
0 thread found 23
0 thread found 29
0 thread found 31
0 thread found 37
0 thread found 41
0 thread found 43
0 thread found 47
0 thread found 53
1 thread found 229
1 thread found 233
1 thread found 239
```

Question 4

4. Modify the program in (3) so that the program prints out the total number of prime numbers found. Check this whether this is correct by running it with exactly 1 thread.

Same as previous one, but to count the total number of prime numbers, reduction clause is used together with sum operator and total_prime_count variable. For efficient program, a function is created with parameter num_threads which holds the total number of threads used in the multithreading. Every time each thread finds a prime number, the total_prime_count (local variable) is increased. At the end, all threads collectively increase the prime counts and at the end the function returns the value.

```
hpc_herald - 4.c

1  #include <stdio.h>
2  #include <omp.h>
3
4  int run_parallel_count(int num_threads) {
5      int i, c;
6      int tid;
7      int total_prime_count = 0;
8      #pragma omp parallel for num_threads(num_threads) default(none) private(i, c, tid) reduction(+:total_prime_count)
9      for (i = 1; i ≤ 1000; i++) {
10         for (c = 2; c ≤ i - 1; c++) {
11             if (i % c == 0)
12                 break;
13         }
14         if (c == i) {
15             tid = omp_get_thread_num();
16             // printf("%d thread found %d\n", tid, i);
17             total_prime_count++;
18         }
19     }
20     return total_prime_count;
21 }
22
23 int main() {
24     int five_threads = 5, one_thread = 1;
25     int five_thread_prime_count = 0, one_thread_prime_count = 0;
26
27     five_thread_prime_count = run_parallel_count(five_threads);
28     one_thread_prime_count = run_parallel_count(one_thread);
29
30     printf("\nTotal prime numbers found using 5 threads: %d", five_thread_prime_count);
31     printf("\nTotal prime numbers found using 1 thread: %d", one_thread_prime_count);
32     return 0;
33 }
```

```
● narayan@narayan workshop % gcc-15 -fopenmp 4.c -o o/4
```

```
● narayan@narayan workshop % o/4
```

```
Total prime numbers found using 5 threads: 168
```

```
Total prime numbers found using 1 thread: 168%
```

Question 5

5. Write an OpenMP program to demonstrate a round-robin message (stored in a global array) passing between three threads.

In this program, three threads are created to pass messages between each other in a round-robin style. To do so, a global array called `messages` is used, where each thread stores the message, it wants to send to the next thread. To control the order of execution, one lock is assigned to each thread. At the beginning, only thread 0 is left unlocked, so it gets to run first, while the other threads remain blocked.

Inside the parallel section, every thread calculates its own ID and determines the ID of the next thread it should send the message to. When a thread's lock becomes free, it means it is that thread's turn. First, the thread receives a message from the previous thread (except for thread 0 during the first loop). After printing the received message, the thread prepares a new message and stores it in the next thread's slot in the global array.

Once the message is stored, the thread unlocks the next thread's lock. This causes the next thread to run, continuing the rotation. Because each thread only runs when its lock is released, the message moves predictably in a circle. This ensures safe and orderly communication between all three threads.

```
hpc_herald - 5.c

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <omp.h>
5
6  char* messages[3] = { NULL, NULL, NULL };
7  omp_lock_t locks[3]; // One lock per thread
8
9  int main() {
10     double start = omp_get_wtime();
11
12     // Initialize all locks
13     for (int i = 0; i < 3; i++) {
14         omp_init_lock(&locks[i]);
15     }
16
17     // Lock all threads except thread 0
18     for (int i = 1; i < 3; i++) {
19         omp_set_lock(&locks[i]);
20     }
21
22     #pragma omp parallel num_threads(3) default(none) shared(messages, locks)
23     {
24         int tid = omp_get_thread_num();
25         int nthreads = omp_get_num_threads();
26         char tmpbuf[100];
27         int dest = (tid + 1) % nthreads;
28
```



```

hpc_herald - 5.c

29     for (int i = 0; i < 10; i++) {
30         // Wait for this thread's turn to execute
31         omp_set_lock(&locks[tid]);
32
33         // RECEIVE phase (skip for thread 0 in first iteration)
34         if (!(i == 0 && tid == 0)) {
35             printf("Thread %d received the message '%s'\n", tid, messages[tid]);
36             free(messages[tid]);
37             messages[tid] = NULL;
38         }
39
40         // SEND phase
41         sprintf(tmpbuf, "Hello from Thread %d!", tid);
42         char* msg = strdup(tmpbuf);
43         messages[dest] = msg;
44         printf("Thread %d sent the message to Thread %d\n", tid, dest);
45
46         // Release next thread
47         omp_unset_lock(&locks[dest]);
48     }
49 }
50
51 // Destroy locks
52 for (int i = 0; i < 3; i++) {
53     omp_destroy_lock(&locks[i]);
54 }
55
56 double end = omp_get_wtime();
57 printf("Time = %f seconds\n", end - start);
58
59 return 0;
60 }
61

```

```
• narayan@narayan workshop % gcc-15 -fopenmp 5.c -o o/5
• narayan@narayan workshop % o/5
Thread 0 sent the message to Thread 1
Thread 1 received the message 'Hello from Thread 0!'
Thread 1 sent the message to Thread 2
Thread 2 received the message 'Hello from Thread 1!'
Thread 2 sent the message to Thread 0
Thread 0 received the message 'Hello from Thread 2!'
Thread 0 sent the message to Thread 1
Thread 1 received the message 'Hello from Thread 0!'
Thread 1 sent the message to Thread 2
Thread 2 received the message 'Hello from Thread 1!'
Thread 2 sent the message to Thread 0
Thread 0 received the message 'Hello from Thread 2!'
Thread 0 sent the message to Thread 1
Thread 1 received the message 'Hello from Thread 0!'
Thread 1 sent the message to Thread 2
Thread 2 received the message 'Hello from Thread 1!'
Thread 2 sent the message to Thread 0
Thread 0 received the message 'Hello from Thread 2!'
Thread 0 sent the message to Thread 1
Thread 1 received the message 'Hello from Thread 0!'
Thread 1 sent the message to Thread 2
Thread 2 received the message 'Hello from Thread 1!'
```

Task 1

Create a product bag in C using an array with fields: productid, name, price, qty, taxable (0/1).

Add at least 10 products.

Create a setting_conf structure with tax_rate.

When program runs:

- Ask for invoice number
- Display product list
- Ask user to choose productid and qty
- Ask user if they want to add another product
 - If yes → show same product list and take input again
 - If no → print bill with: SN Product Name Product Price Qty Total
- At bottom show grand total

```
• narayan@narayan workshop % gcc-15 -fopenmp task1.c -o o/t1
• narayan@narayan workshop % o/t1
Enter invoice number: 123

===== PRODUCT LIST 📋 =====
ID   Name                      Price   Taxable
1    Zanpakuto Replica          49.99   1
2    Shihakusho Outfit          89.50   1
3    Soul Candy Shower          12.75   1
4    Hogyoku Crystal            99.99   1
5    Substitute Badge           25.00   1
6    Quincy Cross                18.99   1
7    Hollows Mask Prop          59.99   1
8    Reiatsu Reader             45.49   1
9    Urahara Hat                 22.00   1
10   Kon Plush Toy              15.99   0

===== PRODUCT LIST 📋 =====

Enter product ID: 1
Enter quantity: 3
Add another item (y/n): y
```

Enter product ID: 2

Enter quantity: 4

Add another item (y/n): n

===== INVOICE =====

Invoice No: 123

SN	Product Name	Price	Qty	Total
----	--------------	-------	-----	-------

1	Zanpakuto Replica	49.99	3	149.97
---	-------------------	-------	---	--------

2	Shihakusho Outfit	89.50	5	447.50
---	-------------------	-------	---	--------

3	Quincy Cross	18.99	7	132.93
---	--------------	-------	---	--------

4	Shihakusho Outfit	89.50	4	358.00
---	-------------------	-------	---	--------

5	Hogyoku Crystal	99.99	4	399.96
---	-----------------	-------	---	--------

6	Shihakusho Outfit	89.50	4	358.00
---	-------------------	-------	---	--------

Grand Total: 1846.36

=====

Task 2

Incremental Requirements (Task 2 Extension)

Add new field `tax_amount` inside the product bill calculation.

Calculate `tax_amount` only if `taxable == 1` using `setting_conf.tax_rate`.

Show `tax_amount` in the bill table alongside:

SN Product Name Product Price Qty Total Tax Amount Final Total ($\text{price} * \text{qty} + \text{tax_amount}$)

At the end of the bill, show:

Subtotal (sum of item totals without tax) Total Tax Discount % (user input 0–99) Discount Amount

Final Payable Amount ($\text{subtotal} + \text{total_tax} - \text{discount}$).

Use multithreading for any calculation step (tax calculation, total calculation, final summary).

```
• narayan@narayan workshop % gcc-15 -fopenmp task2.c -o o/t2
• narayan@narayan workshop % o/t2
Enter invoice number: 234

===== PRODUCT LIST 📌 =====
ID   Name                      Price  Taxable
1    Zanpakuto Replica          49.99    1
2    Shihakusho Outfit          89.50    1
3    Soul Candy Shower          12.75    1
4    Hogyoku Crystal            99.99    1
5    Substitute Badge            25.00    1
6    Quincy Cross                18.99    1
7    Hollows Mask Prop           59.99    1
8    Reiatsu Reader              45.49    1
9    Urahara Hat                 22.00    1
10   Kon Plush Toy               15.99    0

===== PRODUCT LIST 📌 =====

Enter product ID: 1
Enter quantity: 2
Add another item (y/n): y
```

narayan@narayan workshop % o/t2

===== PRODUCT LIST 🍷 =====

Enter product ID: 12

Invalid Product id!

Enter product ID: 8

Enter quantity: 5

Add another item (y/n): n

===== INVOICE =====

Invoice No: 234

SN	Product Name	Price	Qty	Total	Tax Amount	Final Total
1	Zanpakuto Replica	49.99	2	99.98	13.00	112.98
2	Shihakusho Outfit	89.50	10	895.00	116.35	1011.35
3	Hogyoku Crystal	99.99	100	9999.00	1299.87	11298.87
4	Substitute Badge	25.00	4	100.00	13.00	113.00
5	Urahara Hat	22.00	10	220.00	28.60	248.60
6	Kon Plush Toy	15.99	11	175.89	0.00	175.89
7	Reiatsu Reader	45.49	5	227.45	29.57	257.02

Subtotal: 11717.32

Total Tax: 1500.39

Enter Discount Percentage (0-99): 20

Discount 20.00% : 2343.46

Final Payable Amount: 10874.24

=====